

RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique

Shang Liu¹, Wenji Fang, *Graduate Student Member, IEEE*, Yao Lu², Jing Wang³,
Qijun Zhang¹, Hongce Zhang¹, *Member, IEEE*, and Zhiyao Xie¹, *Member, IEEE*

Abstract—The automatic generation of RTL code (e.g., Verilog) using natural language instructions and large language models (LLMs) has attracted significant research interest recently. However, most existing approaches heavily rely on commercial LLMs, such as ChatGPT, while open-source LLMs tailored for this specific design generation task exhibit notably inferior performance. The absence of high-quality open-source solutions restricts the flexibility and data privacy of this emerging technique. In this study, we present a new customized LLM solution with a modest parameter count of only 7B, achieving better performance than GPT-3.5 on all representative benchmarks for RTL code generation. Especially, it outperforms GPT-4 in VerilogEval Machine benchmark. This remarkable balance between accuracy and efficiency is made possible by leveraging our new RTL code dataset and a customized LLM algorithm, both of which have been made fully open-source. Furthermore, we have successfully quantized our LLM to 4-bit with a total size of 4 GB, enabling it to function on a single laptop with only slight performance degradation. This efficiency allows the RTL generator to serve as a local assistant for engineers, ensuring all design privacy concerns are addressed.

Index Terms—Dataset generation, hardware code generation, Verilog, large language model, preference finetuning.

I. INTRODUCTION

IN RECENT years, large language models (LLMs), such as GPT [1], have demonstrated remarkable performance in natural language processing (NLP). Inspired by this progress, researchers have also started exploring the adoption of LLMs in agile hardware design [2]. Many new LLM-based techniques emerge and attract wide attention in 2023. For example, LLM-based solutions are proposed to generate design flow

Received 17 May 2024; revised 16 August 2024; accepted 28 September 2024. Date of publication 17 October 2024; date of current version 21 March 2025. This work was supported in part by the Hong Kong Research Grants Council (RGC) ECS under Grant 26208723; in part by the National Natural Science Foundation of China under Grant 62304192 and Grant 92364102; and in part by the ACCESS—AI Chip Center for Emerging Smart Systems, sponsored by InnoHK funding, Hong Kong, SAR. This article was recommended by Associate Editor M. Poncino. (*Corresponding author: Zhiyao Xie.*)

Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, and Zhiyao Xie are with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, SAR, China (e-mail: sliudx@connect.ust.hk; wfang838@connect.ust.hk; yludf@connect.ust.hk; jwangjw@connect.ust.hk; qzhangcs@connect.ust.hk; eezhiyao@ust.hk).

Hongce Zhang is with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, SAR, China, and also with the Department of Microelectronics Thrust, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou 511455, China (e-mail: hongcezh@ust.hk).

Digital Object Identifier 10.1109/TCAD.2024.3483089

TABLE I
LLM-BASED WORKS ON AUTOMATIC DESIGN RTL (E.G., VERILOG)
GENERATION BASED ON NATURAL LANGUAGE INSTRUCTIONS

Works	New Training Dataset	New LLM Model	Outperform GPT-3.5
Prompt Engineering [10]–[12], [15], [16]	N/A	N/A	N/A
Thakur et al. [14] MG-Verilog [17] Goh et al. [18]	Open-Source	Open-Source	No
VerilogEval [13] ChipNeMo [4] ChipGPT-FT [21]	Closed-Source	Closed-Source	Comparable
BetterV [22]			Yes
RTLCoder	Open-Source	Open-Source	Yes

scripts to control EDA tools [3], [4], design AI accelerator architectures [5], [6], design quantum architectures [7], hardware security assertion generation [8], fix security bugs [9], and even directly generate the target design RTL [4], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20].

Among the above explorations, a promising direction that perhaps attracts the most attention is automatically generating design RTL based on natural language instructions [4], [10], [11], [12], [13], [14], [15], [16], [17], [18]. Specifically, given design functionality descriptions in natural language, LLM can directly generate corresponding hardware description language (HDL) code,¹ such as Verilog, VHDL, and Chisel from scratch. Compared with well-explored *predictive* machine learning (ML)-based solutions in EDA [23], such *generative* methods benefit the hardware design and optimization process more directly. This LLM-based design generation technique can potentially revolutionize the existing HDL-based VLSI design process, relieving designers from the tedious HDL coding tasks.

Table I summarizes existing works in LLM-based design RTL generation. Some works [10], [11], [12], [15], [16] focus on prompt engineering methods based on commercial LLMs like GPT, without proposing new datasets or models for RTL code generation. As we will discuss later, reliance on commercial LLM tools limits in-depth research exploration

¹Most existing works focus on generating design RTL in Verilog code. In this work, we also choose Verilog, while the method should be general and applicable to other HDL types like VHDL. We will use terms *RTL code* and *Verilog code* interchangeably.

and incurs serious privacy concerns in industrial IC design scenarios. Thakur et al. [14] generated a large unsupervised training² dataset by collecting Verilog-based projects from online resources like GitHub, then fine-tuning its own model. However, this unsupervised dataset is quite unorganized with a mixture of code and text. Evaluations on a third-party benchmark [12] show that the performance of its fine-tuned model is still inferior to commercial tools like GPT-3.5. The VerilogEval [13] from the NVIDIA research team proposes its own labeled training dataset and benchmark, then fine-tunes its own new model. This may be the first noncommercial model that claims comparable performance with GPT-3.5, but according to their authors, neither the training dataset nor fine-tuned LLM model will be released to the public in the near future [13]. Goh et al. [18] proposed an open-source dataset with Verilog instruction-code samples. However, their instructions have only module name information, without code functionality descriptions. A recently released dataset MG-Verilog [17] proposed a multigrained dataset consisting of descriptions of various level details with code to enhance the model's instruction-following ability. But the model performance of [17] and [18] also cannot outperform GPT-3.5. Besides these customized RTL-generation solutions, according to our study, all other software code (e.g., Python) generation models like CodeGen2 [24], StarCoder [25], and Mistral [26] are significantly inferior to GPT-3.5 in this RTL generation task.

Compared with solutions based on closed-source commercial LLM tools like GPT, the open-source LLM solution is vitally important from both research and application perspectives: 1) for research purposes, obviously, closed-source commercial tools prevent most in-depth studies and customizations of this emerging technique and 2) for realistic applications, users of commercial LLM tools unavoidably have data privacy concerns, since all instructions have to be uploaded to LLM providers like OpenAI. The privacy concern is especially critical in the VLSI design industry, where information leakage of intellectual property (IP) or key technical innovations can seriously hurt the competitive advantage of users' companies. In comparison, each user's own local LLM developed based on an open-source solution can eliminate all privacy concerns and also ensure a reliable service.

However, as mentioned, high-performance open-source RTL generation models are currently unavailable. According to our study, a major challenge is the unavailability of high-quality circuit design data for training: 1) organized design data is mostly owned by semiconductor companies, who are almost always unwilling to share design data and 2) design data directly collected online is messy and unorganized, either leading to inferior model performance or requiring prohibitive human efforts to clean the dataset.

In this work, we finally fill this gap with our new open-source LLM solution named *RTLCode*.³ To the best of our knowledge, it is the first noncommercial and open-source LLM method that clearly outperforms GPT-3.5 in design RTL code generation. We validate this on two representative benchmarks [12], [13] and observe consistent trends. To build this *RTLCode*, we first propose an automated data generation flow and have generated a dataset with over 27 000 instruction-code samples for the RTL generation task.

RTLCode obviously achieves state-of-the-art tradeoffs between performance and efficiency. Besides demonstrating unprecedented RTL generation correctness in noncommercial solutions, it only has 7 billion (B) parameters and can be trained with only four consumer-level GPU cards. After further quantizing the parameters to 4 bits, the *RTLCode*-4bit takes only 4 GB of memory and can work on a laptop with limited accuracy loss. As a result, our open-source lightweight *RTLCode* solution is accessible to almost every research group and it is friendly to be implemented and improved by researchers. The contributions of *RTLCode* can be summarized below.

- 1) Targeting Verilog code generation, we propose an automated flow to generate a large dataset with over 27 thousand diverse Verilog design problems and answers. It addresses the serious data availability challenge in IC design-related tasks, and its potential applications are not limited to LLMs. The LLM directly trained on it can already achieve comparable accuracy with GPT-3.5.
- 2) We introduce a new LLM training scheme based on code quality feedback. It further boosts the ultimate model performance to outperform GPT-3.5, being comparable with GPT-4. We further revised the training process from an algorithm perspective to reduce its GPU memory consumption. The training process only requires 4 commercial-level GPU cards.
- 3) We designed *RTLCode* to be a lightweight solution with only 7B parameters. After quantizing its parameters into 4 bits, it takes only 4 GB of memory, allowing it to serve as a local assistant for engineers without privacy concerns.
- 4) *RTLCode* has been fully open-sourced, including our data generation flow, complete generated dataset, LLM training algorithm, and the fine-tuned model. Considering *RTLCode*'s lightweight property and low-hardware barrier, it allows anyone to easily replicate and further improve based on our existing solution.

II. AUTOMATIC DATASET GENERATION

In this work, we first propose a new automated training dataset generation flow. Based on this flow, we have generated over 27 thousand training samples, with each sample being a pair of design description instruction (i.e., model input) and the corresponding reference RTL code (i.e., expected

²Most customized LLM solutions (including *RTLCode*) are developed by fine-tuning pretrained LLMs based on a training dataset about the specific task. In this article, we use the terms *training* and *fine-tuning* interchangeably.

³Our *RTLCode* has been fully open-source at <https://github.com/hkust-zhiyao/RTL-Coder>. It includes the data generation flow, the complete generated training data set, the model training flow, and the final fine-tuned models (based on both Mistral and DeepSeek).

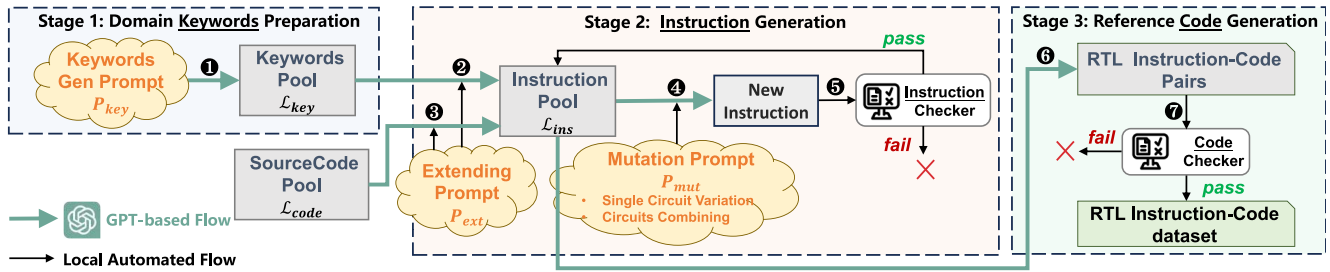


Fig. 1. Our proposed automated training dataset generation flow.

Prompt:
Please act as a professional Verilog designer. List the categories and examples of blocks that are frequently used in Verilog coding tasks.

Fig. 2. Example of Prompt P_{key} in ❶.⁴

model output). The instruction can be viewed as the input question for LLMs, describing the desired circuit functionality in natural language. The reference code is the expected answer from LLMs, implementing the circuit functionality in Verilog code. We observe that these generated training samples exhibit high diversity and complexity in the RTL-generation domain, encompassing a diverse spectrum of difficulty levels.

We build this automated generation flow by taking full advantage of the powerful general text generation ability of the commercial tool GPT. Please notice that GPT is only used for dataset generation in this work, and we adopt GPT-3.5 in this data generation task. The automated dataset generation flow is illustrated in Fig. 1, which includes three stages: 1) RTL domain keywords preparation; 2) instruction generation; and 3) reference code generation. We designed several general prompt templates to control GPT generating the desired outputs in each stage.

A. Stage 1: Keywords Preparation

The first stage of our data generation flow targets preparing RTL domain keywords for subsequent stages. At process ❶ shown in Fig. 1, we request GPT to generate keywords related to digital IC design (i.e., commonly used logic components) based on a set of prompts P_{key} . We obtain a keyword pool $\mathcal{L}_{key}$ with hundreds of digital design keywords.

Specifically, in this process ❶, to collect a comprehensive range of RTL design task topics, we utilize a tree-like structure with multiple branches to issue queries to GPT. We first prompt GPT at the root node to provide categories and examples of frequently used block keywords in RTL design as Fig. 2 illustrated. The response from GPT has a tree structure that consists of some subfields as Fig. 3 shows. With the response, we could use the categories and examples as branches to continue prompting GPT for more design keywords within each topic. For example, we can use scripts

⁴We use *red* text boundary to denote GPT *input* examples, and *green* text boundary to denote GPT *output* examples in this work. Please notice that some *green* GPT *output* in this data generation flow are instructions, which will be the input of LLMs.

Response:

1. **Combinational Logic Blocks:** Arithmetic blocks (adders, subtractors, multipliers, dividers)...
2. **Sequential Logic Blocks:** Shift Registers (serial-in/serial-out, serial-in/parallel-out, parallel-in/serial-out, parallel-in/parallel-out), Sequence detectors...
3. **Finite State Machines (FSMs):** Mealy FSM, Moore FSM, One-hot FSM, Gray-code FSM...
4. **Digital Signal Processing (DSP) Blocks:** Filters (FIR, IIR), Fast Fourier Transform (FFT)...
5. **Communication Protocol Blocks**

Fig. 3. GPT response example to Prompt P_{key} in ❶.

to ask GPT about more types of the block “multiplier,” it will return more specific design names, such as “Booth multiplier, Wallace tree multiplier, etc.” After this process, we obtain hundreds of keywords related to RTL design in the Keywords pool $\mathcal{L}_{key}$.

Besides using our introduced keywords generation flow, we believe manually collecting initial Verilog instructions from existing IP families is also feasible and can help to produce a comprehensive keyword pool, as long as users devote manual efforts to collecting the IP document resources and checking their coverage and diversity.

This obtained keyword pool with about 350 instances will be extended into around 5K instruction samples in process ❷ and these instructions will be further mutated and combined with each other and those obtained from source code in stage 2 to generate more new descriptions.

B. Stage 2: Instruction Generation

The second stage targets generating sufficient instructions based on the initial keywords and Verilog source code. At process ❷, we extend existing keywords from $\mathcal{L}_{key}$ to complete design instructions. Specifically, we randomly sample one or two keywords from $\mathcal{L}_{key}$ each time, combined with prompts P_{ext} , and feed them into GPT. The output is a complete RTL design instruction.

In addition to keyword-based instruction generation in process ❷, we also propose to generate instructions based on existing source code collected by us, as shown in process ❸. This is partially inspired by the work of [27]. By providing GPT with either part or a complete Verilog code $\mathcal{L}_{code}$ collected by [14], we can inspire it to create a related Verilog design problem. By adopting this new ❸ together with ❷, we

You should **create a task** that only requires one Verilog module related to the given topic.
Here is an example for you.

[Given Topic]

UART transmitter

[Instruction]

Create a Verilog module for a UART transmitter that can send data at a baud rate of 9600. The module should have a single input for the data to be transmitted (8 bits) and ...

```
module uart_transmitter(
    input clk, // system clock
    input reset_n, // active low asynchronous reset
    input [7:0] data_in, // data to be transmitted
    output reg tx, // UART transmit pin
    output reg tx_busy // UART transmit busy signal
);
```

Fig. 4. Example of Prompt $P_{\text{ext}}^{\text{key}}$ in ②.

further enhance the diversity of our dataset by utilizing a vast and varied collection of source code.

Processes ② and ③ help generate the initial design instruction pool $\mathcal{L}_{\text{ins}}$ based on our customized prompt P_{ext} . Two types of prompt P_{ext} are proposed for processing $\mathcal{L}_{\text{key}}$ and $\mathcal{L}_{\text{code}}$, denoted as $P_{\text{ext}}^{\text{key}}$ and $P_{\text{ext}}^{\text{code}}$, respectively. As shown in Fig. 4, our prompt $P_{\text{ext}}^{\text{key}}$ in process ② adopts the few-shot prompting technique, which means we provide an example of the question (i.e., keyword) and answer (i.e., instruction) in the input prompt. Fig. 5 shows an example of GPT's corresponding response. As for the prompt $P_{\text{ext}}^{\text{code}}$ used in process ③, an example of prompt and the response of GPT are provided in Figs. 6 and 7. The prompt $P_{\text{ext}}^{\text{code}}$ asks GPT to convert the given Verilog code snippet to the corresponding description instruction.

For process ③, instructions obtained through code snippets from $\mathcal{L}_{\text{code}}$ focus more on describing detailed circuit behavior (such as the specific behavior in different cycles of a signal). On the other hand, the instructions obtained from the keyword pool $\mathcal{L}_{\text{key}}$ in process ② tend to provide an overall summary of the circuit's functionality in a high-level manner. For example, based on the keyword "traffic light" in the keyword pool, a series of traffic light circuit instructions can be generated. These overall descriptions can hardly be obtained solely through code snippets if the signal name is not related to something like "red_light" or "yellow_light." The instructions generated through keywords not only describe circuit behavior but also link to commonly used digital design terminologies and higher-level abstracts. Hence, both initial instruction generation methods are crucial and complementary. They describe circuit behavior from different levels of description (highly abstract and detailed to the behavior of each signal).

In the extension process of ② and ③, the prompt for generating the instruction responses should be carefully designed. We provide Fig. 6 for the reader's reference. The instructions returned by GPT should contain detailed and unambiguous requirements to ensure that LLM can generate only one functionally unique Verilog code based on this instruction.

[Given Topic]

Pulse width modulator (PWM).

[Instruction]

Please act as a professional Verilog designer. Your task is to create a Verilog module that implements a pulse width modulator (PWM). You need to design a PWM module that takes a clock signal, a reset signal, and a user-defined duty cycle value as inputs, and outputs a PWM signal. The duty cycle value should be an integer ranging from 0 to 100, representing the percentage of the PWM signal being high during each cycle.

Your module should meet the following **requirements**:

- The PWM signal should have a fixed frequency, determined by a parameter called PWM_FREQUENCY. You can assume that the input clock frequency is 100 MHz.
- The duty cycle of the PWM signal should be adjustable by changing the duty_cycle input value.
- When the rst input is high, the PWM output should be reset to low, and the duty cycle should start from the beginning of the next cycle when the reset signal goes low.
- The PWM output should transition from low to high at the beginning of each cycle and should stay high for a duration proportional to the duty_cycle input value.
- You can assume that the input clock signal is a perfect square wave with a 50% duty cycle, and the reset signal is synchronous to the input clock. The solution should be synthesizable and follow good coding practices.

```
module PWM(
    input clk,
    input rst,
    input [7:0] duty_cycle, // 8-bit duty cycle
    value (0-100)
    output pwm_out
);
```

Fig. 5. GPT response example (green) to Prompt $P_{\text{ext}}^{\text{key}}$ when given topic: Pulse width modulators (PWM) (red) in ②.

Prompt:

Please gain inspiration from the following random code snippet to create a high-quality Verilog coding problem.

Verilog code snippet for inspiration:

```
if ((C_REG_CONFIG == 32'h00000001) || (C_REG_CONFIG == 32'h00000008)) begin reg [C_DATA_WIDTH-1:0] m_payload_i; reg [C_DATA_WIDTH-1:0] skid_buffer... has_valid_storage_i = 1'b1; // Clear the value if it's set and Master side completes the transaction but we don't have a new slave side.
```

The Verilog coding problem should be completely self-contained, providing all the contextual information one needs to understand and write code to implement this module. Assume common programming knowledge, but ensure that any specific context, variables, or code snippets pertinent to this description are explicitly included. You should describe the behavior of the circuit in detail.

Fig. 6. Example of Prompt $P_{\text{ext}}^{\text{code}}$ in ③.

What is more, the design complexity should be limited so that the corresponding Verilog code can be implemented in hundreds of code lines.

After generating the initial instruction pool $\mathcal{L}_{\text{ins}}$ with hundreds of initial instructions, we will iteratively use mutation methods to significantly augment the scale and complexity

Response:

This Verilog module implements a configurable slave interface with three different modes of operation. The module has the following inputs and outputs...The module has three modes of operation, selected by the value of C_REG_CONFIG: Mode 1 (C_REG_CONFIG == 32'h00000001 or C_REG_CONFIG == 32'h00000008): This mode is a pass-through mode where the slave interface forwards data from the master interface to the slave interface and vice versa...

```
module configurable_slave_interface (
    input  ACLK,
    input  ARESET,
    input  [31:0] C_REG_CONFIG,
    input  C_DATA_WIDTH = 32,
    input  S_VALID...)

```

Fig. 7. GPT response example to Prompt $P_{\text{ext}}^{\text{code}}$ in ③.

The **rewritten task** should achieve different circuit functionality but require similar methods or components in the given instruction. Here is one example for you.

[Given Instruction]

Please act as a professional Verilog coder. Create a module that implements a finite state machine (FSM), used for 5-bit sequence detection.

[New Instruction]

Please act as a professional Verilog designer. Create a module that implements a finite state machine (FSM), used for 5-bit sequence detection, but with a twist. The module should detect two different 5-bit sequences, "01010" and "10101", and output a signal indicating which sequence was detected. The output signal should be '0' for "01010" and '1' for "10101". The FSM should transition to the initial state whenever a reset signal is asserted. Implement the FSM using a Mealy machine, where the output is a function of both the current state and the input.

Fig. 8. Example of Prompt P_{mut}^s in ④.

of this pool.⁵ At ④, we use P_{mut} to apply two types of mutation operations on instructions sampled from the design instruction library $\mathcal{L}_{\text{ins}}$. The process ④ would check every new design instruction using a set of rules and only passed valid instructions are added to $\mathcal{L}_{\text{ins}}$. We cover more details of this iterative process below.

For the mutation operation in ④, we propose two types of prompts P_{mut}^s and P_{mut}^c , performing single circuit variation and circuits combination, respectively.

- 1) The P_{mut}^s focuses on changing the single circuit module by requiring different functionalities while keeping the same hardware type, or implementing the same functionality but using different hardware. Fig. 8 is an example of P_{mut}^s .
- 2) The P_{mut}^c focuses on combining two circuit designs together. They are combined by either adding control logic to select using one of the two functionalities (i.e.,

⁵The scale and complexity in this context describe the target circuit. Circuit scale can be reflected by the total number of logic gates in the circuit after implementation. Circuit complexity can be reflected in its functionality complexity, which may be reflected by the number of lines in its implementation. Such metrics are also used in prior benchmarking works [12]. As an example, in Fig. 9, we illustrate how combining a binary counter and a comparator can result in a circuit with a larger area and more complex functionality.

You should create a **new task** based on the two given instructions. The output of one module could be the input of the other. Here is an example for you.

[Given instructions]

1. Create a module that implements a **binary counter**...
2. Create a module that implements a **comparator**...

[New instruction]

Create a module that implements a **binary counter** that counts from 0 to 15, ..., Additionally, include a **comparator** that compares the value of the counter to a given value, and outputs a signal indicating whether the counter value is greater than or equal to the given value.

Fig. 9. Example of Prompt P_{mut}^c in ⑤.

parallel combination) or feeding one output into the other's input (i.e., serial combination). Fig. 9 gives a sample of prompt P_{mut}^c .

It is important to note that we need to carefully design the instruction generation prompt considering the following two aspects in the mutation process.

- 1) Basic instruction validity.⁶ For example, GPT may provide an instruction of implementing a physical temperature regulator or a speedometer, which are not directly related to Verilog coding.
- 2) Level of circuit behavior detail for LLM to generate correct code. Other studies [14], [17] have also indicated that the level of circuit description has a significant impact on the code generation quality. When descriptions are overly vague, LLMs trained on this dataset struggle to align the functionality of the code with the general instruction. Conversely, if descriptions are too detailed, focusing on intricate RTL circuit specifics, the RTL generation effectively becomes a form of "code translation," which can also not boost the general generative abilities of trained LLMs.

Moreover, we request GPT to generate its reasoning steps (i.e., how it analyzes the code generation task step-by-step). These reasoning steps further enhance the detailed information of our instruction pool. In addition to incorporating these principles in the query prompt, we also use the one-shot technique (provide a query-response sample for GPT's reference) to demonstrate our requirements. As long as following these principles and adopt basic prompt engineering techniques, we believe other alternative prompts may achieve similar response quality.

For the instruction checking in ⑤, we automatically check the correctness and diversity of each new instruction generated in the mutation process, based on a series of rules. The rules include 1) invalid-word checking and 2) similarity measurement with existing $\mathcal{L}_{\text{ins}}$. For example, in invalid-word checking, if words, such as "image" and "text," are detected, it indicates that GPT has likely provided a problem that cannot be implemented solely with RTL code. As for similarity rate checking, we calculate the similarity between the new instruction and existing $\mathcal{L}_{\text{ins}}$. The new instruction is discarded if the similarity is above a certain threshold since it cannot add enough new information to $\mathcal{L}_{\text{ins}}$. Such checking of instructions

⁶"Validity" here refers that the instruction can be implemented solely by Verilog code, requiring no other tools or languages.

in ⑤ is imperfect, but it is fully automated and accurate enough to generate an ultimate instruction pool $\mathcal{L}_{\text{ins}}$, including over 50 000 instructions.

C. Stage 3: Reference Code Generation

The third stage targets generating the reference code. In the third stage, as shown in ⑥, we feed each instruction from $\mathcal{L}_{\text{ins}}$ into GPT, generating the corresponding reference design code as the solution. Specifically, we will ask GPT to provide at least five answers. After that, in ⑦, we will evaluate these answers using a code checker. In this work, we adopt an automated syntax checker to verify the correctness of RTL code syntax. Only syntax-correct design code can be kept. If all five answers fail the syntax checking, this instruction will be discarded. Finally, only valid instruction-code pairs are saved as our dataset. Ideally, process ⑦ should also check whether the functionality of the generated RTL code is consistent with the instruction, but currently generating testbenches for functionality verification cannot be automated. Similar to the checker in stage 2, this imperfect automated checking can already filter out the most serious mistakes in the dataset.

D. Dataset Generation Flow Discussion

After going through all three proposed stages, we generate the ultimate training dataset named RTLCoder-27K with more than 27 000 instruction-code data samples. In this section, we will discuss some important topics about our proposed data generation flow and the output dataset. Multiple experiments are conducted to provide more insights about our RTLCoder dataset.

Statistics of Keyword Pool: In the keyword generation process, we adopt the hierarchical prompting structure to query GPT, resulting in around 350 keywords. Experienced engineers have recognized the keyword pool as covering a vast majority of commonly used circuit design problems. We also adopted GPT to analyze the keyword types and illustrate the categories and distribution of this keyword pool in Fig. 10(a).

Comparison With Other RTL Dataset: We compare RTLCoder with other recent RTL code datasets [13], [14], [17], [18] in the following. In Thakur et al. [14], the training dataset only consists of code without instructions. Such code-only datasets do not align well with the RTL generation tasks based on natural language instructions. After removing exactly the same duplicated code in the dataset, there are approximately 25K data entries. In VerilogEval [13], each data entry comprises an instruction-code pair, totaling 8.5K entries in their dataset. However, their dataset is not open-sourced. In the MG-Verilog dataset [17], there are 11K instances with each one consisting of descriptions at various levels of detail and corresponding code. In the Goh et al. [18], there are a total of 60K samples. However, their instruction only provides the module name information without any functionality descriptions. The performance of LLMs [13], [14], [17], [18] trained on these four datasets will be presented in Table III in Section IV and all cannot clearly outperform GPT3.5. RTLCoder significantly outperforms all

TABLE II
DIVERSITY SCORES OF FOUR REPRESENTATIVE FINE-TUNING INSTRUCT DATASETS. THERE ARE THREE VERILOG DATASETS, INCLUDING OUR RTLCoder-27K, GOH ET AL. [18], AND MG-Verilog [17]. WE ALSO EVALUATE A WIDELY USED SOFTWARE (MAINLY PYTHON) DATASET, MAGICoder-OSS-INSTRUCT-75K [27]. THE LOWER VALUE OF CR AND CR: POS, THE HIGHER-DATASET DIVERSITY. RTLCoder-27K EXHIBITS A SATISFACTORY LEVEL OF DIVERSITY COMPARED WITH THE OTHER THREE DATASETS

	RTLCoder-27K	Goh et al. [18]	MG-Verilog [17]	Magicoder-OSS-Instruct-75K [27]
CR	4.41	5.27	5.80	4.02
CR: POS	7.61	10.1	9.16	6.67

these models, indicating the overall quality of our open dataset.

Evaluation of Design Type Distribution: We also utilized GPT for annotating the categories of the generated 27K-dataset. The circuit type distribution statistics is depicted in Fig. 10(b). The distribution statistics for the two benchmarks, RTLLM-1.1 [12] and VerilogEval [13] are also plotted in Fig. 10(c) and (d), respectively. The generated dataset consists of a wide range of circuit designs, including arithmetic units, signal processing units, encryption units, etc. It is important to note that a single data sample may implement functionalities of multiple circuit types, and we only adopt one circuit type to represent each instruction. As the majority of circuits contain submodules like multiplexers, decoders, and selectors, this may result in a relatively higher proportion of the data path type.

Evaluation of Dataset Diversity: To further check the diversity of our proposed training dataset, RTLCoder-27K, we utilized two diversity measures: 1) compression ratios (CRs) and 2) part-of-speech CR (CR: POS) which are suggested the best-lexical diversity metrics by [28] among homogenization score (BERTScore), self-BLEU, homogenization score (ROUGE-L), N -gram diversity score, hypergeometric distribution, etc. CR is calculated utilizing text compression algorithms which can identify redundancy in the whole contents. We constructed a file D containing all the text data in RTLCoder-27K following [28]. Subsequently, we compressed the entire dataset using gzip. The CR is as follows:

$$\text{CR}(D) = \frac{\text{size of } D \oplus}{\text{compressed size of } D \oplus}.$$

The CR-POS can capture the repeated syntactic redundancy by compressing the part-of-speech (POS) tag sequences of the original text. We also followed [28] and used NLTK POS tagger and the Penn Treebank set of 36 tags to extract the tag sequences [28].

The results are shown in Table II. Additionally, we also provide the CR and CR:POS values for three representative fine-tuning datasets: 1) Goh et al. [18]; 2) MG-Verilog [17]; and 3) Magicoder-OSS-Instruct-75K [27] for comparison. A smaller CR and CR-POS value indicates a higher level of diversity in the dataset. We observe that RTLCoder-27K exhibits similar diversity to the widely used Python benchmark and higher diversity compared to the Verilog code datasets.

Imperfection in Data Functionality Correctness: Our dataset generation flow ensures the syntax correctness of all instruction-code data samples, but cannot guarantee that every

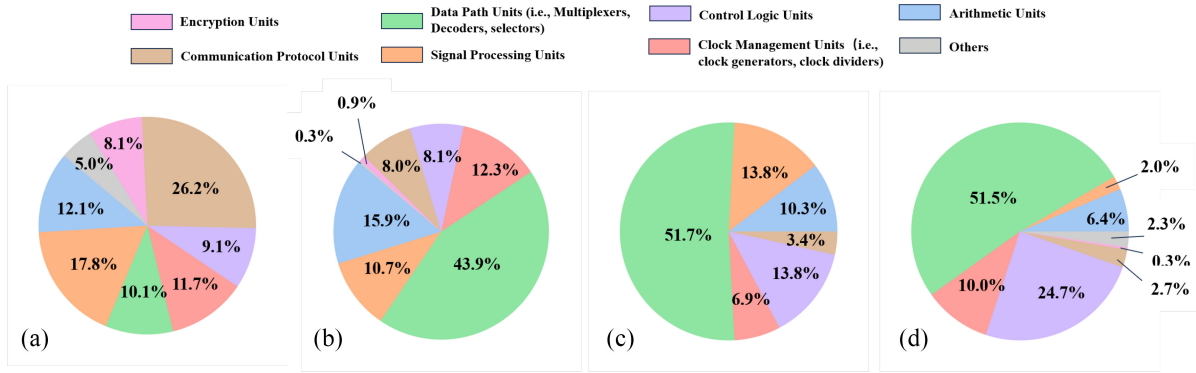


Fig. 10. Circuit type distribution of keyword pool, the obtained dataset RTLCode-27K, RTLLM-1.1 [12], and VerilogEval [13]. (a) Keyword pool. (b) Our 27K. (c) RTLLM-1.1. (d) VerilogEval(Machine + Human).

data sample is functionally correct (i.e., code implements the same functionality as described in instruction). This is because automatically checking the functionality correctness of training samples is inherently a very challenging task. Functionality checking for the Verilog code is practically hardware verification, which has been studied for decades, relies on human engineers, and is difficult to get guaranteed results. Although the code in our proposed dataset has only undergone syntax checks, finetuning the model using this dataset can still lead to improved model performance on the benchmarks as Table III shows in Section IV. This indirectly demonstrates the contribution of the proposed dataset.

Relation Between RTLCode and GPT-3.5: An interesting observation is that, although we generate our training dataset based on GPT-3.5, RTLCode turns out to outperform the GPT-3.5 baseline on representative benchmarks [12], [13]. One important reason is that, for each instruction, we have employed a syntax checker to evaluate the reference code generated based on GPT-3.5. Therefore, among all correct and incorrect code from GPT-3.5, we filter out the obviously incorrect ones and retain the largely correct ones for training RTLCode. This process can be viewed as a refinement of GPT-3.5's Verilog generation capabilities.

III. NEW TRAINING SCHEME INCORPORATING CODE QUALITY FEEDBACK

Besides the new training dataset, we propose a new LLM training scheme that incorporates code quality scoring. It significantly improves the RTLCode's performance on the RTL generation task. Also, we revised the training process from the algorithm perspective to reduce the GPU memory consumption of this new training method, allowing implementation with limited hardware resources.

A. Existing Supervised Training on LLMs

This part will first introduce the existing supervised training method for LLMs. Then we will further discuss its limitations in RTL generation tasks. Suppose we have a training data dataset $\{x_i, y_i\}$ for $i = 1, \dots, N$, where x_i represents an design instruction, y_i represents the corresponding correct reference code. Each sample of data will be split into a sequence of tokens by certain rules during the preprocessing process. In

this article, we use $x_i = \{x_i^t\}$ and $y_i = \{y_i^t\}$ for $t = 1, 2, \dots, T$ to represent the tokenized sequence.

LLMs generate a sequence by continuously predicting the next token based on the already generated previous ones. For a decoder-only language model, which is the mainstream LLM architecture, the probability of producing the next token depends only on the previous output tokens and the input instruction. We denote the probability of generating the t th token r_t (r_t can be any single token in the vocabulary) as $P_\pi(r_t | x_i, y_i^{<t})$ where π represents the model parameters and $y_i^{<t}$ denotes the already generated previous tokens $\{y_i^1, \dots, y_i^{t-1}\}$. Then the log probability of generating the whole sequence can be written as: $\sum_{t=1}^T \log P_\pi(y_i^t | x_i, y_i^{<t})$.

In the existing training method, maximum likelihood estimation (MLE) is commonly used to find the best parameters π that maximize the log probability. The training flow is shown in Fig. 11(a). The loss is usually defined as below

$$loss_{mle} = - \sum_{t=1}^T \log P_\pi(y_i^t | x_i, y_i^{<t}).$$

However, there exists a phenomenon named *exposure bias* [29], [30]. Since the above sequence generation is autoregressive, which means the model always predicts the next token based on its own generated previous ones $r_i^{<t}$ rather than the reference tokens $y_i^{<t}$. Therefore, even though the probability of producing y_i^t is high when given $y_i^{<t}$ in the training, it can still result in a huge deviation from the reference code in the generation process.

We have also observed this phenomenon in our experiments. After the supervised training, the qualities of multiple generated code candidates for the same instruction may diverse greatly in the performance aspect. They can include correct code while at the same time, including many low-quality answers. Some candidates exhibit serious nonsense duplication.⁷

To alleviate the *exposure bias* phenomenon, we suggest that in addition to the reference code y_i , the model's generation should also be considered in the training process. Since the

⁷We notice that this duplication could not be simply dealt with by adding repetition penalty to the decoding process like other works in natural text generation. Because some correct RTL design code also contain similarly repetitive expressions.

TABLE III

PERFORMANCE COMPARISON OF RTL CODE GENERATORS ON VERILOGEVAL BENCHMARK [13] AND RTLLM BENCHMARK [12]. THE TOP SCORES RANKED FIRST, SECOND, AND THIRD IN EACH COLUMN ARE MARKED IN GREEN, BLUE, AND RED, RESPECTIVELY. RTLCodeR OUTPERFORMS GPT-4 ON EVALMACHINE OF [13]. IT IS ONLY SECOND TO GPT-4 ON THE OTHER BENCHMARKS (EVALHUMAN OF [13] AND RTLLM [12]), OUTPERFORMING GPT-3.5, AND ALL OTHERS

Model Type	Evaluated Model	Num of Params	VerilogEval Benchmark [13] (using pass@k metric)						RTLLM V1.1 [12] (using pass@5 metric)	
			Eval-Machine (%)			Eval-Human (%)			Syntax-VCS (%)	Func (%)
			k=1	k=5	k=10	k=1	k=5	k=10		
Closed-Source Baseline	GPT-3.5	N/A	46.7	69.1	74.1	26.7	45.8	51.7	89.7	37.9
	GPT4	N/A	60.0	70.6	73.5	43.5	55.8	58.9	100	65.5
	ChipNeMo* [4]	13B	43.4	N/A	N/A	22.4	N/A	N/A	N/A	N/A
	VerilogEval* [13]	16B	46.2	67.3	73.7	28.8	45.9	52.3	N/A	N/A
	BetterV* [22]	7B	64.2	75.4	79.1	40.9	50.0	53.3	N/A	N/A
Open-Source Baseline	Codegen2 [24]	16B	5.00	9.00	13.9	0.90	4.10	7.25	72.4	6.90
	Starcode [25]	15B	46.8	54.5	59.6	18.1	26.1	30.4	93.1	27.6
	Thakur et al. [14]	16B	44.0	52.6	59.2	30.3	43.9	49.6	86.2	24.1
	MG-Verilog et al. [17]	7B	52.7	58.5	60.9	N/A	N/A	N/A	N/A	N/A
	Goh et al. [18]	7B	40.6	48.4	54.4	N/A	N/A	N/A	N/A	N/A
Base Model	Mistral-7B-v0.1 [26]	7B	36.9	48.8	57.4	4.49	12.6	18.6	72.4	20.7
	DeepSeek-Coder-6.7b [33]	6.7B	54.1	63.8	67.5	30.2	42.2	46.2	89.6	34.5
Less Training Data (10K Samples)	RTLCodeR-Mistral-10k	7B	56.5	66.6	69.4	31.7	42.2	46.5	86.2	34.5
	RTLCodeR-DeepSeek-10k	6.7B	55.3	70.4	76.2	36.7	47.0	50.4	79.3	37.9
Direct Training	RTLCodeR-Mistral-Direct	7B	58.9	70.0	74.1	34.4	42.3	45.1	89.7	41.4
	RTLCodeR-DeepSeek-Direct	6.7B	59.8	73.6	77.2	39.1	48.3	51.3	86.2	44.8
RTLCodeR	RTLCodeR-Mistral-4bit	7B * 4bit	59.5	72.2	76.9	33.8	42.3	47.1	86.2	41.4
	RTLCodeR-DeepSeek-4bit	6.7B * 4bit	56.5	73.2	78.4	37.5	50.5	55.5	93.1	37.9
	RTLCodeR-Mistral	7B	62.5	72.2	76.6	36.7	45.5	49.2	96.6	48.3
	RTLCodeR-DeepSeek	6.7B	61.2	76.5	81.8	41.6	50.1	53.4	93.1	48.3

*We cannot directly evaluate VerilogEval [13], ChipNeMo [4] and BetterV [22] on RTLLM Benchmark due to closed-source models. We fully understand and respect the authors' privacy concerns. The accuracy values of VerilogEval [13], ChipNeMo [4], BetterV [22], GPT-3.5, and GPT-4 on the VerilogEval Benchmark [13] are directly cited from the original publication [4], [13], [22].

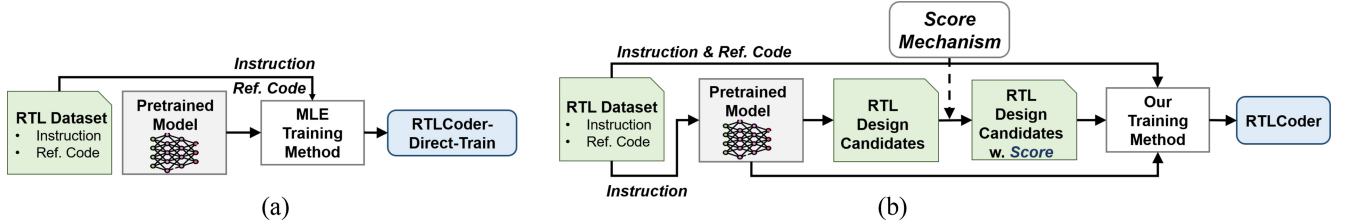


Fig. 11. Comparison between (a) existing MLE-based LLM training flow and (b) our proposed LLM training flow.

generation may be different from the reference code, it is necessary to introduce a scoring mechanism to judge the quality of generated candidates. We will give our detailed solution in Section III-B.

B. Our Proposed Training Method

Our proposed training scheme is illustrated in Fig. 11(b). For each instruction, we will now collect multiple code candidates generated by the initial pretrained model. Then, we pack these candidates and the original reference code y_i together as $y_i = \{y_{i,k}\}$, $k = 1, 2, \dots, K$, where K represents the number of generated code for one instruction. Next, all these candidates will be scored by the scoring mechanism $R(x_i, y_{i,k})$ which could be a syntax checker or unit test for functionality check. We will then obtain a set of score $\mathbf{z}_i =$

$\{z_{i,k}\}$, $k = 1, 2, \dots, K$, denoting the quality for the code sample $\{y_{i,k}\}$. In the training process, we aim to make the model learn to assign relatively higher-generation probabilities to answers with higher scores. In this way, the model not only learns from the reference code, but also from the new information introduced by the quality score feedback.

The conditional log probability (length-normalized) of generating the entire code $y_{i,k}$ is commonly written as

$$p_{i,k} = \frac{\sum_t \log P_\pi(y_{i,k}^t | x_i, y_{i,k}^{<t})}{\|y_{i,k}\|}.$$

We calculate $p_{i,k}$ for all code candidates $\mathbf{y}_i = \{y_{i,k}\}$, $k = 1, 2, \dots, K$, then we normalize these $p_{i,k}$ values using a *softmax* function, defining the probability of each code being selected

Algorithm 1 Training Scheme Using Gradients Splitting

Input: The single data sample $\{x_i, y_i, z_i\}$. Model forward function $s_{i,k} = f_\pi(x_i, y_i, z_i, k)$. Loss calculation function $L_\pi(s_i, z_i)$. GPU affordable batch size J . Model parameters w .
Output: The derivative of the loss with respect to model parameters: g_i .

- 1: Group the sample $\{x_i, y_i, z_i\}$ for $k = 1, 2, \dots, K$ into Q parts based on batch size J .
- 2: initialize empty vector list *temp*. Initialize the gradients $g_i = 0$.
- 3: **for** $q \in Q$ **do**
- 4: Calculate $s_{i,k} = f_\pi(x_i, y_i, z_i, k)$, for $k \in q$.
- 5: Empty the computation graph
- 6: Calculate $loss = L_\pi(s_i, z_i)$ // $s_i = \{s_{i,k}\}$ for $k = 1, \dots, K$
- 7: Backward process: $temp_k = \partial loss / \partial s_{i,k}$, for $k = 1, \dots, K$
- 8: **for** $q \in Q$ **do**
- 9: Calculate $s_{i,k} = f_\pi(x_i, y_i, z_i, k)$, for $k \in q$
- 10: Backward process: $g_i = g_i + \sum_{k \in q} temp_k \partial s_{i,k} / \partial w$
- 11: Empty the computation graph
- 12: Return g_i

as

$$s_{i,k} = \frac{e^{p_{i,k}}}{\sum_{\tau=1}^K e^{p_{i,\tau}}}.$$

This $s_{i,k}$ reflects the model's tendency to output the k th code candidate, with higher probabilities indicating a greater likelihood that the model will generate it.

To encourage the model to assign higher-probability scores to high-quality code, we can define a new loss function term

$$loss_{compare} = \sum_{z_{i,k} < z_{i,\tau}} \max(s_{i,k} - s_{i,\tau} + \lambda, 0)$$

where λ is a threshold value.

To provide an intuitive explanation of this loss function term, we provide a simple example. Suppose we have the i th instruction and only two code candidates with initial selection probability $s_{i,1}$ and $s_{i,2}$ with $s_{i,1} + s_{i,2} = 1$ and $s_{i,1} > s_{i,2}$. But the first candidate has a lower-quality score, i.e., $z_{i,1} < z_{i,2}$. Then the positive loss would drive model parameters to update until the model assigns a new set of $s_{i,1}^*$ and $s_{i,2}^*$ so that $s_{i,2}^* - s_{i,1}^* \geq \lambda$ is satisfied.

It is worth noting that this loss only depends on the relative scores among multiple code candidates, so it can still be used when answer quality cannot be precisely quantified. Finally, We define the total loss as

$$loss = loss_{compare} + loss_{mle}.$$

C. Reduced Memory by Splitting Gradients

Directly calculating our new *loss* function even with 1 batch size would still require forwarding all code candidates in a sample at once to maintain all the activation values. This will lead to the $O(K)$ space complexity and make the GPU memory consumption prohibitively high in many LLM training scenarios.

We propose a gradient-splitting approach for model training based on quality score from an algorithm perspective. It can achieve a $O(1)$ space complexity as illustrated in Algorithm 1. The gradients of *loss* with respect to w can be computed as below

$$\frac{\partial loss}{\partial w} = \sum_k \frac{\partial loss}{\partial s_{i,k}} \frac{\partial s_{i,k}}{\partial w}.$$

The property of the chain rule indicates that we can decompose the gradient updates into several parts. Assume J is the maximum allowable batch size for GPU consumption. We divide the K candidates into Q groups based on the batch size J . First, we pass these groups through the forward function separately and collect the obtained s_i values as lines 1–5 illustrate. In the second step, we calculate the loss function and compute the derivative of the loss with respect to s_i in lines 6 and 7, storing the temporary results in vector *temp*. In the third step, we perform the forward operation on the original Q groups again and for each forward operation, the obtained $s_{i,k}$ is multiplied by $temp_k$ in a dot product, followed by a backward pass to accumulate the gradient in lines 9–12.

IV. EXPERIMENTAL RESULTS**A. Evaluation Benchmark and Metric**

To evaluate the performance of Verilog code generation, there are two representative benchmarks VerilogEval [13] and RTLML [12].

The VerilogEval [13] benchmark consists of two parts, EvalMachine and EvalHuman, each including more than 100 RTL design tasks. We follow the original paper [13] and use the widely adopted *pass@k* metric in code generation tasks

$$pass@k = E_i \left(1 - \frac{C_{n-c_i}^k}{C_n^k} \right)$$

where n is the total number of trials for each instruction and c_i is the number of correct code generations for task i . We set $n = 20$ in this experiment. If any code in the k trials could pass the test, then this task is considered to be addressed and the *pass@k* metric reflects the estimated proportion of design tasks that could be solved.

The RTLML V1.1 [12] benchmark contains 29 RTL design tasks at a larger design scale. We mostly follow the testing method in the original paper [12], but further proposes two slightly different metrics for evaluating syntax correctness, using either Synopsys VCS [31] or design compiler [32]. They are denoted as Syn-VCS and Syn-DC, respectively.

- 1) For the Syn-VCS metric, VCS not only requires the design to comply with the Verilog syntax rules, but also requires that the interface of the design correspond to the testbench, so that the circuit can be simulated.

- 2) For the Syn-DC metric, DC requires the design to be physically synthesizable. The functionality result is obtained by VCS simulation.

We calculate the scores of the design syntax part and design functionality part separately. In both parts, following the original benchmark [12], each task is counted as success as

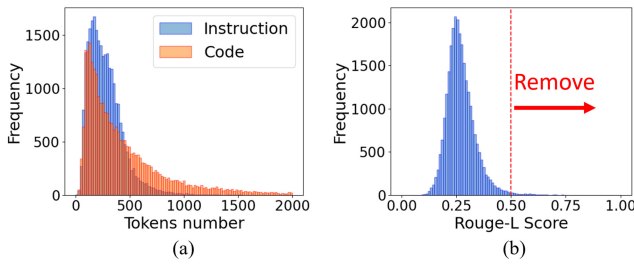


Fig. 12. Training dataset analysis. (a) Tokens number distribution of instruction and code part. (b) Similarity measurement between training dataset and two benchmarks based on Rouge-L metric.

long as *any* of five trials passes the test. This can be interpreted as pass@5 metric.

In the generation process, we set $top_p = 0.95$ and $temperature = \{0.2, 0.5, 0.8\}$. For all tested models (i.e., baselines, RTLCoder, and ablation studies), we evaluate all 3 *temperature* conditions and report the best performance for each model.

B. Examine Training Set for Fair Evaluation

To ensure a fair evaluation of our proposed RTLCoder, before training, we explicitly examined the similarity between samples in our proposed training dataset and those test cases in benchmarks [12], [13], then we get rid of our training samples that are similar to test cases during the training process.

To measure the similarity between two text sequences, we employed the Rouge-L metric, which is a widely used similarity calculation scheme in the LLM domain, such as by OpenAI [1]. The Rouge-L score $\in [0, 1]$, with values closer to 1 indicating higher similarity between the two sequences. For each instruction-code concatenated sample in the training dataset, we computed its Rouge-L value with all test cases in the benchmarks. In addition, we also separately analyzed the distribution of token counts for instructions and code in the dataset. The resulting statistic is in Fig. 12.

From Fig. 12(a), we can see that a sample that consists of one instruction and one code candidate is generally within 2048 token length. So we can set 2048 as the max length in our finetuning. In Fig. 12(b), we observed that the majority of training samples in the dataset have a low overlap compared with the benchmark, with Rouge-L scores < 0.3 . However, there are still a small number of samples with higher similarity. To ensure fair evaluation of the RTLCoder, we get rid of all training samples with Rouge-L values > 0.5 which counts about 100 samples.

C. Model Training

Based on our generated dataset with 27K instruction-code pairs, we choose the latest Mistral-7B-v0.1 [26] and DeepSeek-Coder-6.7b [33] as the basic pretrained model for finetuning. In all experiments, we opted for the Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.999$, and learning rate $\gamma = 1e-5$, while abstaining from the use of weight decay. Concurrently, we established a context length of 2048 and a global batch size of 256. We trained the model on only four consumer-level RTX

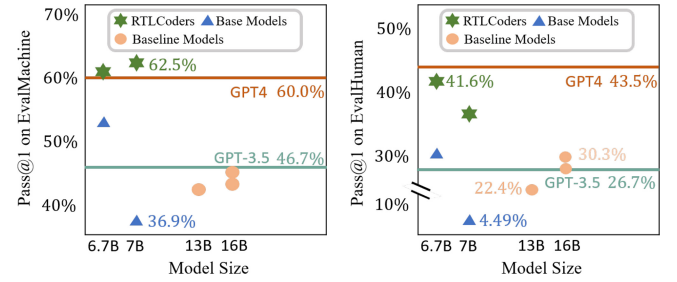


Fig. 13. Visualization of key accuracy comparisons from Table III, selecting pass@1 metric on EvalMachine and EvalHuman of [13]. RTLCoder includes both RTLCoder-Mistral and RTLCoder-DeepSeek. The baseline models include Thakur et al. [14], ChipNeMo [4], and VerilogEval [13].

4090 GPUs (24 GB each), each of which could only afford 2×2048 context length using DeepSpeed stage-2 [34]. Under the hardware constraint, the training is impossible without the proposed gradient-splitting method.

To implement our proposed training scheme, we first generated 3 code candidates for each instruction using the pretrained model with the Beam search method. Then we use Pyverilog [35] as the syntax checker to score the code candidates. Specifically, we assigned a full score (i.e., 1) for the reference code from the dataset and those candidates who can pass the syntax check. For those who failed syntax checks, we used the Rouge-L metric to assign the code similarity between the candidate and reference code as its score.

In addition, considering GPU memory consumption is a crucial factor that limits the applicability of LLMs, based on quantization methodologies [36], we further quantize the parameters of the obtained RTLCoder into 4 bits, generating RTLCoder-DeepSeek-4bit and RTLCoder-Mistral-4bit, consuming only 4-GB memory.

D. Experiment Results Overview

Table III summarizes the comparison of all relevant RTL generation solutions, including commercial models GPT3.5/GPT4, models customized for Verilog generation [13], [14], [22], software code generators [24], [25], [26], [33], our proposed RTLCoder and quantized version RTLCoder-4bit, and ablation studies of RTLCoder. In addition, we further visualize key results on VerilogEval benchmark in Fig. 13.

In the VerilogEval benchmark [13], for both EvalHuman and EvalMachine categories, RTLCoder-DeepSeek scores 61.2 and 41.6, respectively. It clearly outperforms GPT-3.5 and is only inferior to GPT-4 among all the models in EvalHuman. Specifically, in the EvalMachine part, RTLCoder-DeepSeek and RTLCoder-Mistral even outperforms GPT4 by an absolute value of 1.2% and 2.5%. A similar trend can be observed in the RTLLM benchmark V1.1 [12]. RTLCoder is also second only to GPT-4. In summary, RTLCoder outperforms GPT-3.5 and all noncommercial baseline models in all metrics on both benchmarks. It is surprising that the lightweight RTLCoder with only 7 billion parameters could achieve such impressive accuracy despite its smaller size.

TABLE IV
DETAILED SYNTAX AND FUNCTIONALITY EVALUATION RESULTS USING SAMPLING GENERATION METHOD IN RTL2LLM V1.1 [12]

Design	GPT-3.5			GPT-4			Thakur et al. [14]			StarCoder[25]			RTLCoder-Mistral-4bit			RTLCoder-Mistral		
	Syn-VCS	Syn-DC	Func	Syn-VCS	Syn-DC	Func	Syn-VCS	Syn-DC	Func	Syn-VCS	Syn-DC	Func	Syn-VCS	Syn-DC	Func	Syn-VCS	Syn-DC	Func
accu	2	2	✓	5	5	✓	4	4	✗	3	4	✗	5	5	✗	4	4	✗
adder_8bit	3	3	✓	4	4	✓	3	3	✓	2	4	✗	5	5	✓	5	5	✓
adder_16bit	1	0	✗	3	3	✓	3	4	✓	2	3	✗	0	0	-	3	3	✗
adder_32b	0	0	-	2	2	✓	1	0	✗	1	3	✗	1	0	✗	1	0	✗
adder_pipe_64b	5	5	✗	5	5	✓	0	0	-	0	0	-	1	1	✗	3	2	✗
multi_booth_8b	5	2	✗	5	5	✗	3	3	✗	4	3	✗	5	5	✓	5	5	✓
multi_16b	5	0	✓	5	5	✓	3	3	✗	3	4	✗	4	2	✗	5	5	✓
multi_pipe_4b	0	0	-	2	2	✗	1	0	✗	3	1	✗	4	1	✗	2	0	✗
multi_pipe_8b	2	0	✗	5	5	✗	3	1	✗	2	3	✗	0	0	-	2	0	✗
div_8bit	3	1	✗	5	1	✗	0	1	-	3	0	✗	3	1	✗	4	1	✗
div_16bit	4	0	✗	5	4	✓	1	2	✗	1	1	✗	0	0	-	0	0	-
JC_counter	5	5	✗	5	5	✗	3	3	✗	4	5	✗	5	5	✓	5	4	✓
right_shifter	4	4	✓	5	5	✓	0	2	-	3	3	✓	5	5	✓	5	5	✓
synchronizer	5	5	✓	4	4	✓	4	4	✓	5	5	✓	4	4	✓	5	5	✓
counter_12	5	5	✓	5	5	✓	2	4	✓	2	4	✓	5	5	✓	5	5	✓
freq_div	5	5	✓	5	5	✓	4	4	✓	4	4	✗	5	5	✓	5	3	✓
signal_gen	5	5	✓	5	5	✓	4	5	✗	4	4	✗	5	5	✗	5	5	✗
serial2parallel	4	4	✗	5	5	✓	4	4	✗	4	4	✗	5	3	✗	5	3	✗
parallel2serial	2	2	✗	5	5	✓	1	2	✗	3	4	✓	3	3	✗	3	2	✓
pulse_detect	4	4	✗	5	3	✗	4	3	✗	3	3	✗	5	5	✗	2	2	✗
edge_detect	5	5	✓	5	5	✓	4	5	✓	3	4	✓	4	2	✓	5	4	✓
FSM	5	4	✗	5	2	✗	4	4	✗	5	5	✗	4	4	✗	5	5	✗
width_8to16	4	3	✓	5	5	✓	4	1	✗	3	4	✗	5	5	✓	5	4	✓
traffic_light	4	0	✗	4	3	✓	5	2	✗	5	3	✗	4	0	✓	4	3	✓
calendar	5	5	✗	5	5	✓	2	1	✗	5	4	✓	1	0	✗	5	5	✗
RAM	4	0	✓	5	2	✓	5	5	✓	2	0	✓	3	0	✓	3	0	✓
asyn_fifo	0	0	-	3	2	✗	0	0	-	0	0	-	0	2	-	1	3	✗
ALU	2	0	-	5	4	-	2	2	✗	1	0	✗	2	1	✗	1	0	✗
PE	5	5	✓	5	5	✓	3	3	✗	3	5	✓	1	1	✓	5	5	✓
Success rate	89.7%	65.5%	11/29	100%	100%	19/29	86.2%	86.2%	7/29	93.1%	82.8%	8/29	86.2%	75.9%	12/29	96.6%	79.3%	14/29

Furthermore, we validate the effectiveness of our proposed dataset and algorithm through an ablation study. The RTLCoder-Mistral-Direct and RTLCoder-DeepSeek-Direct are directly trained with the existing method mentioned in Fig. 11(a). Using our training dataset, they can already significantly outperform the base model and even GPT-3.5 on part of these indexes. Then the RTLCoders trained with our proposed training scheme further outperform those using Direct training method on all benchmarks, indicating that our training method greatly further improves the model performance.

In addition, although the quantized model RTLCoder-DeepSeek-4bit shows a slight performance degradation compared to the original model, it is still superior to GPT-3.5 on the VerilogEval benchmark and comparable to it on RTL2LLM V1.1 with only 4-GB size. Such RTLCoder-4bit can work on a simple laptop, allowing it to serve as a local assistant for engineers, addressing privacy concerns.

We also randomly selected 10K samples from the 27K training dataset to finetune the base models and obtained RTLCoder-Mistral-10k and RTLCoder-DeepSeek-10k, respectively. Compared with the two models, RTLCoders trained on a 27K dataset are clearly superior on all metrics. Increasing the size of the training dataset and enhancing its diversity clearly further improves the model performance.

As for the pretrained model selection, we can see that different base model also has a significant impact on the performance of the fine-tuned model. On one hand, RTLCoder-DeepSeek slightly outperforms RTLCoder-Mistral in accuracy

on most benchmarks. This trend is consistent with the base model's relative accuracy (i.e., DeepSeek outperforms Mistral in most benchmarks). On the other hand, the inference speed of RTLCoder-Mistral is considerably faster than RTLCoder-DeepSeek, largely because of the grouped query attention and rolling buffer KV cache techniques used in Mistral.

E. Experiment Results in Detail

To further examine the performance in detail, for both benchmarks [12], [13], we report RTLCoder's performance on each individual design case in both syntax and functionality correctness.

We list the test results of RTLCoder-Mistral and available baseline models on the RTL2LLM V1.1 benchmark for each design task in Table IV. Given five trials of generation, here we counted the number of passed cases in terms of Syn-VCS, Syn-DC, and Functionality. As introduced, for both syntax and functionality, we count one success if any of the five trials pass the test. Generally, Syn-VCS is easier to pass than Syn-DC.

We further inspect the wrong answers in Table IV. We observed that the overall code structures of wrong answers from GPT-3.5, GPT-4, and RTLCoder-Mistral exhibit no obvious mistakes, despite the functionality incorrectness. In comparison, the code generated by other open-source baselines occasionally contains obviously redundant content or deviates considerably from the given description. In terms of syntax, we observed that both GPT and RTLCoder-Mistral frequently

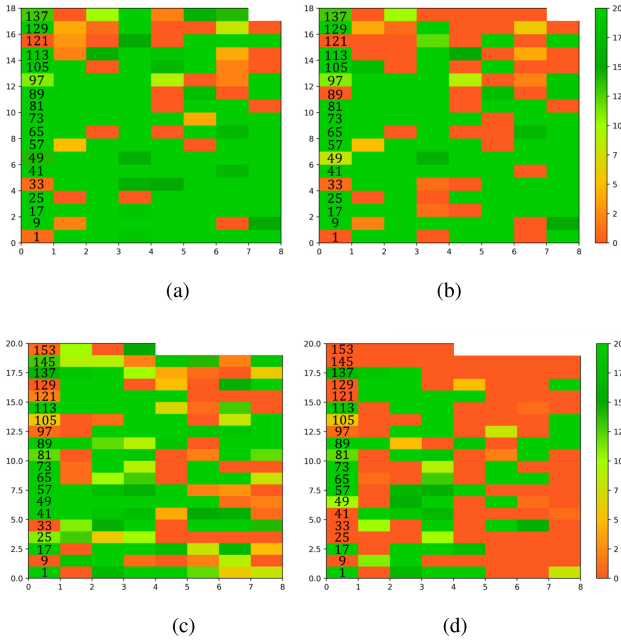


Fig. 14. Detailed syntax and functionality results of RTLCode-Mistral on VerilogEval Benchmark [13], reporting EvalMachine and EvalHuman separately. Each subfigure has eight columns, and thus cell at (i, j) represents the $((j-1) \times 8 + i)$ th task. The color of each cell indicates the count of correct cases among 20 trials. EvalMachine contains 143 tasks, so the last one cell is empty. EvalHuman contains 156 tasks, so the last four cells are empty. (a) EvalMachine syntax. (b) EvalMachine functionality. (c) EvalHuman syntax. (d) EvalHuman functionality.

assign 0 directly to 2-D arrays, resulting in syntax errors. Regarding functionality, we noticed that for more complex combinational logic circuits, such as multi_pipe_4bit and multi_pipe_8bit, and sequential logic circuits like pulse_detect and FSM, some of the logical behaviors described in the instructions are not adequately captured by all LLM solutions, leading to functional errors.

The RTLCode-Mistral's results on VerilogEval Benchmark are reported in Fig. 14. Each cell in the image represents one design case, with color indicating the number of successful ones among all 20 trials. There are eight columns in each image. The location of cell (i, j) represents the $((j-1) \times 8 + i)$ th design case in the provided description file. So we used white cells to fill the cells in the last row (18th row for EvalMachine and 20th row in the EvalHuman) that do not correspond to a design task.

During the process of generating text sequences, the model continuously repeats the behavior of predicting the next token. For all models in our experiment, we adopt the sampling method, which randomly selects the next token from the vocabulary dictionary based on the probability distribution. Here, we further add an ablation study based on the beam search method. A beam of the top “beam size” subsequences with the highest-generation probabilities is maintained and updated during the generation process. We conduct experiments using beam search method with a beam size 5 on RTLLM V1.1 for RTLCode-Mistral and open source baselines. The results are shown in Table V. The accuracies of all methods drop after adopting beam search. RTLCode-Mistral

is still superior to all the open-source baselines with beam search.

We also investigate the performance of the RTLCode fine-tuned on the Verilog task compared to its pretrain models on other code-generation tasks. Table VI shows pass@1 results on Verilog tasks from VerilogEval-Human [13], python tasks from HumanEval+ [37], Cpp, and Sh programming tasks from [38]. RTLCode-DeepSeek shows a significant improvement over its base model in the Verilog task and performs similarly to the base model in the other three programming tasks. RTLCode-Mistral, compared to its base model, also shows better performance in the Verilog task but experiences more performance degradation in the Cpp and Sh benchmarks. This observation indicates that different pretrain models when fine-tuned on the same specific task, exhibit varying degrees of forgetting when applied to other benchmarks. Overall, such a degradation in other programming tasks does not affect our target application (i.e., circuit RTL generation).

V. LIMITATION AND FUTURE WORK

The results of GPT4 on VerilogEval-Human and RTLLM 1.1 are very impressive in Table III which indicates the significant superiority of GPT4 to all the other reported models. In our opinion, GPT-4, the most powerful large-scale model by OpenAI to date, outperforms existing open-source models for mainly three main reasons.

- 1) The size of the pretraining dataset used by GPT-4 is orders of magnitude larger than our adopted pretrained models. The generation of a huge dataset that enables high-performance LLM consumes a significant amount of manpower and resources;
- 2) The GPT-4 model is much larger in scale than any existing open-source model. The scaling law [39] suggests that as the model parameters increase, the model's generalization and learning capabilities also improve; and
- 3) The alignment technique, such as RLHF [40], employed by GPT-4 can enhance the quality of the model's outputs, reducing incorrect answers, which is also one of the motivations for us proposing the scoring-based training approach in this article.

Given limited resources, we believe there are several key strategies for the open-source model to outperform GPT4 in specific tasks.

- 1) The diversity and coverage of the dataset can be further improved. Due to limited manual efforts during dataset generation, the training set primarily relies on guiding GPT to create instructions. To further enhance the coverage of the dataset for Verilog design problems, manual inspection and creation for high-quality data sample generation can be conducted.
- 2) The checking of functional correctness of the Verilog code should be further explored if possible, using either manpower or a reliable automated verification process. Although our fine-tuned models have a significant

TABLE V

ABLATION STUDY OF DIFFERENT DECODING METHODS IN RTL-LM V1.1 BENCHMARK [12]. THE RESULT OF THE SAMPLING DECODING METHOD IS ADOPTED AND REPORTED IN TABLE III

Model	Sampling decoding [used in experiment]			Beam search decoding [for ablation study]		
	Syn-VCS	Syn-DC	Func	Syn-VCS	Syn-DC	Func
Thakur et al. [14]	86.2	86.2	24.1	69.0	51.7	17.2
StarCoder[25]	93.1	82.8	27.6	58.6	58.6	17.2
RTL-Coder-Mistral-4bit	86.2	75.9	41.4	75.9	65.5	31.0
RTL-Coder-Mistral	96.6	79.3	48.3	75.9	72.4	37.9

TABLE VI

PASS@1 RESULTS OF THE TRAINED RTLCODER AND TWO BASE MODELS ON DIFFERENT PROGRAMMING TASKS

	Verilog	Python	C++	Sh
Mistral-7B-v0.1	4.49	25.2	30.1	9.07
DeepSeek-Coder-6.7b-v1	30.2	66.8	63.9	36.3
RTL-Coder Mistral	36.7	25.3	21.4	1.99
RTL-Coder DeepSeek	41.6	66.7	63.0	32.6

performance improvement and even outperform GPT-3.5, reliable filtering based on functionality checking can certainly further boost the model's generation ability.

- 3) The automated functionality checking for hardware is challenging. A possible solution is to apply LLMs, such as GPT, to also help generate verification assertions based on instructions [41]. Then the code combined with assertions can be verified by verification platforms, such as Cadence Jasper. However, the correctness of assertions, since they are also automatically generated by LLMs, is not guaranteed either. This may result in wrongly filtering out many correct samples.
- 4) The scoring-based training scheme can be improved by applying automated functionality checking. This technique can hopefully enhance the model's instruction alignment capability.

VI. CONCLUSION

This work proposes a new LLM solution named RTLCoder for RTL code generation, achieving state-of-the-art performance in noncommercial solutions and outperforming GPT-3.5. We contribute a new data generation flow and a complete dataset with over 27 thousand instruction-answer samples, addressing the serious data availability problem in hardware-design-related tasks. Also, we contribute a new training scheme based on design quality scoring. It greatly boosts the model's performance. Importantly, RTLCoder has been fully open-sourced. RTLCoder's lightweight property and low-hardware barrier allow anyone to easily replicate and further improve based on our existing solution. We expect more brilliant LLM-based solutions in this agile hardware design direction.

REFERENCES

[1] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[2] L. Chen et al., "The dawn of AI-native EDA: Promises and challenges of large circuit models," 2024, *arXiv:2403.07257*.

[3] Z. He et al., "ChatEDA: A large language model powered autonomous agent for EDA," in *Proc. MLCAD Workshop*, 2023, pp. 1–14.

[4] M. Liu et al., "ChipNeMo: Domain-adapted LLMs for chip design," 2023, *arXiv:2311.00176*.

[5] Y. Fu et al., "GPT4AIGChip: Towards next-generation AI accelerator design automation via large language models," 2023, *arXiv:2309.10730*.

[6] Z. Yan, Y. Qin, X. S. Hu, and Y. Shi, "On the viability of using LLMs for SW/HW co-design: An example in designing CiM DNN accelerators," 2023, *arXiv:2306.06923*.

[7] Z. Liang et al., "Unleashing the potential of LLMs for quantum computing: A study in quantum architecture design," 2023, *arXiv:2307.08191*.

[8] R. Kande et al., "LLM-assisted generation of hardware assertions," 2023, *arXiv:2306.14027*.

[9] B. Ahmad, S. Thakur, B. Tan, R. Karri, and H. Pearce, "Fixing hardware security bugs with large language models," 2023, *arXiv:2302.01215*.

[10] K. Chang et al., "ChipGPT: How far are we from natural language hardware design," 2023, *arXiv:2305.14019*.

[11] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," 2023, *arXiv:2305.13243*.

[12] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTL-LM: An open-source benchmark for design RTL generation with large language model," 2023, *arXiv:2308.05345*.

[13] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating large language models for Verilog code generation," 2023, *arXiv:2309.07544*.

[14] S. Thakur et al., "Benchmarking large language models for automated Verilog RTL code generation," in *Proc. DATE*, 2023, pp. 1–7.

[15] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, "AutoChip: Automating HDL generation using LLM feedback," 2023, *arXiv:2311.04887*.

[16] M. Nair, R. Sadhukhan, and D. Mukhopadhyay, "Generating secure hardware using ChatGPT resistant to CWEs," *Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2023/212*, 2023.

[17] Y. Zhang, Z. Yu, Y. Fu, C. Wan, and Y. C. Lin, "MG-Verilog: Multi-grained dataset towards enhanced LLM-assisted Verilog generation," 2024, *arXiv:2407.01910*.

[18] E. Goh, M. Xiang, I. Wey, and T. H. Teo, "From English to ASIC: Hardware implementation with large language model," 2024, *arXiv:2403.07039*.

[19] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie, "OpenLLM-RTL: Open dataset and benchmark for LLM-aided design RTL generation," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2024, pp. 1–9.

[20] M. Li, W. Fang, Q. Zhang, and Z. Xie, "SpecLLM: Exploring generation and review of VLSI design specification with large language model," 2024, *arXiv:2401.13266*.

[21] K. Chang et al., "Data is all you need: Finetuning LLMs for chip design via an automated design-data augmentation framework," 2024, *arXiv:2403.11202*.

[22] Z. Pei, H.-L. Zhen, M. Yuan, Y. Huang, and B. Yu, "BetterV: Controlled Verilog generation with discriminative guidance," 2024, *arXiv:2402.03375*.

[23] M. Rapp et al., "MLCAD: A survey of research in machine learning for CAD keynote paper," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 10, pp. 3162–3181, Oct. 2022.

[24] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou, "CodeGen2: Lessons for training LLMs on programming and natural languages," 2023, *arXiv:2305.02309*.

[25] R. Li et al., "StarCoder: May the source be with you!" 2023, *arXiv:2305.06161*.

[26] A. Q. Jiang et al., "Mistral 7B," 2023, *arXiv:2310.06825*.

[27] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, "Magicoder: Source code is all you need," 2023, *arXiv:2312.02120*.

[28] C. Shaib, J. Barrow, J. Sun, A. F. Siu, B. C. Wallace, and A. Nenkova, "Standardizing the measurement of text diversity: A tool and a comparative analysis of scores," 2024, *arXiv:2403.00553*.

[29] Y. Liu, P. Liu, D. Radev, and G. Neubig, "BRIO: Bringing order to abstractive summarization," 2022, *arXiv:2203.16804*.

[30] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Proc. NeurIPS*, 2015, pp. 1–9.

[31] (Synopsys, Inc., Sunnyvale, CA, USA). VCS® *Functional Verification Solution*. (2021). [Online]. Available: <https://www.synopsys.com/verification/simulation/vcs.html>

- [32] (Synopsys, Inc., Sunnyvale, CA, USA). *Design Compiler® RTL Synthesis*. (2021). [Online]. Available: <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/design-compiler-nxt.html>
- [33] D. Guo et al., "DeepSeek-coder: When the large language model meets programming—The rise of code intelligence," 2024, *arXiv:2401.14196*.
- [34] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2020, pp. 3505–3506.
- [35] S. Takamaeda-Yamazaki, "Pyverilog: A python-based hardware design processing toolkit for Verilog HDL," in *Proc. 11th Int. Symp. Appl. Reconfig. Comput.*, 2015, pp. 451–460.
- [36] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training compression for generative pretrained transformers," 2022, *arXiv:2210.17323*.
- [37] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Proc. 37th Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 21558–21572.
- [38] L. Ben Allal, N. Muennighoff, L. Kumar Umapathi, B. Lipkin, and L. von Werra, "A framework for the evaluation of code generation models," 2022. [Online]. Available: <https://github.com/bigcode-project/bigcode-evaluation-harness>
- [39] J. Kaplan et al., "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.
- [40] Y. Bai et al., "Training a helpful and harmless assistant with reinforcement learning from human feedback," 2022, *arXiv:2204.05862*.
- [41] W. Fang et al., "AssertLLM: Generating and evaluating hardware verification assertions from design specifications via multi-LLMs," 2024, *arXiv:2402.00386*.



Shang Liu received the B.E. degree in automation science and electrical engineering from Beihang University, Beijing, China, in 2023. He is currently pursuing the Ph.D. degree with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong.

His research interests include agile VLSI design methodologies and artificial intelligence.



Wenji Fang (Graduate Student Member, IEEE) received the B.Eng. degree from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2021, and the M.Phil. degree in microelectronics from The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China, in 2024. He is currently pursuing the Ph.D. degree with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong.

His research interests include electronic design automation and VLSI design verification.



Yao Lu received the B.E. degree from the School of Electronic Science and Engineering, Southeast University, Nanjing, China, in 2020, and the master's degree from the School of Microelectronics, Fudan University, Shanghai, China, in 2023. She is currently pursuing the Ph.D. degree with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong.

Her current research interests focus on machine learning applications in EDA.



Jing Wang received the B.S. degree in electrical information engineering from Peking University, Beijing, China, in 2022, and the master's degree in artificial intelligence from the Department of Statistics and Actuarial Science, The University of Hong Kong, Hong Kong, in 2023. He is currently pursuing the Ph.D. degree with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong.

His research interests include agile VLSI design methodologies and artificial intelligence.



Qijun Zhang received the B.Eng. degree from Tongji University, Shanghai, China, in 2022. He is currently pursuing the Ph.D. degree with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong.

His research interests include computer architecture and electronics design automation.



Hongce Zhang (Member, IEEE) received the B.S. degree in microelectronics from Shanghai Jiao Tong University, Shanghai, China, in 2015, and the Ph.D. degree from the Electrical and Computer Engineering Department, Princeton University, NJ, USA, in 2021.

He is currently an Assistant Professor with the Department of Microelectronics Thrust, Function Hub of Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China, and is also affiliated with the Electronic and Computer

Engineering Department, The Hong Kong University of Science and Technology, Hong Kong. His research interests include formal verification and hardware model checking.



Zhiyao Xie (Member, IEEE) received the B.Eng. degree from The City University of Hong Kong, Hong Kong, in 2017, and the Ph.D. degree from Duke University, Durham, NC, USA, in 2022.

He is an Assistant Professor with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong. His research interests include machine learning algorithms for EDA and VLSI design.

Dr. Xie has received multiple prestigious awards, including the IEEE/ACM MICRO 2021 Best Paper

Award, the ACM SIGDA SRF Best Research Poster Award 2022, the ASP-DAC 2023 Best Paper Award, the ACM Outstanding Dissertation Award in EDA 2023, the EDAA Outstanding Dissertation Award 2023, and the 2023 Early Career Award from Hong Kong Research Grants Council.